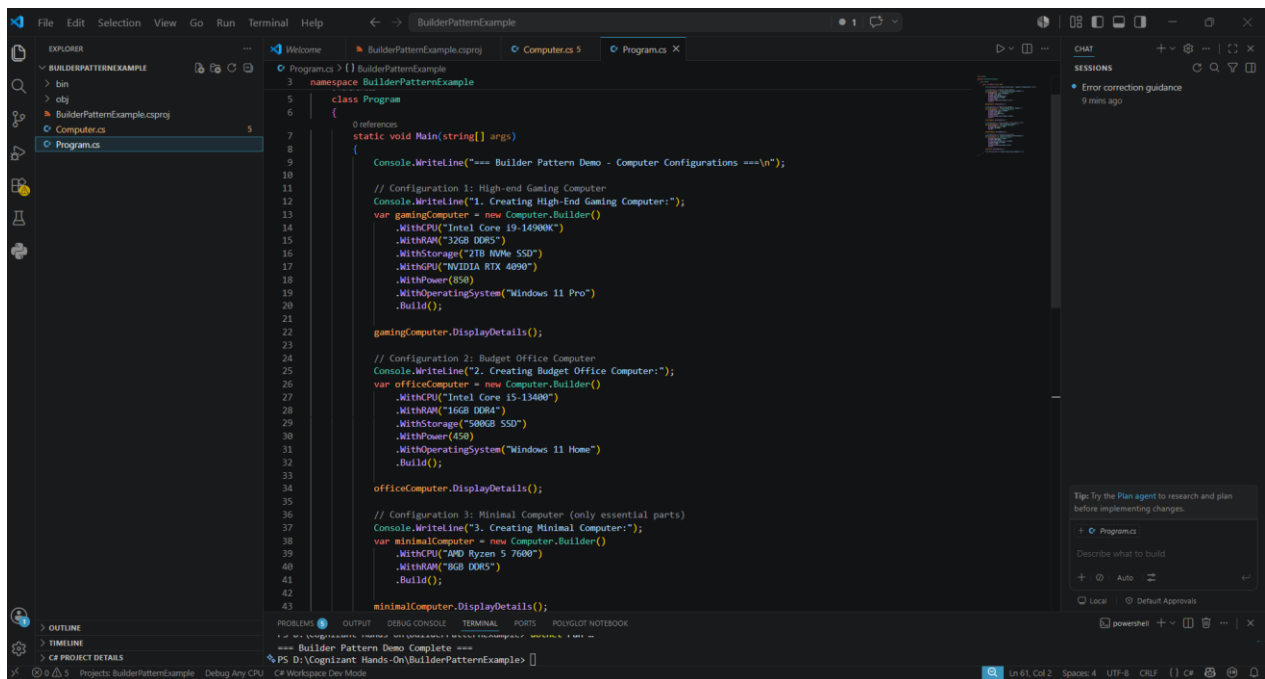


Hands – On Report

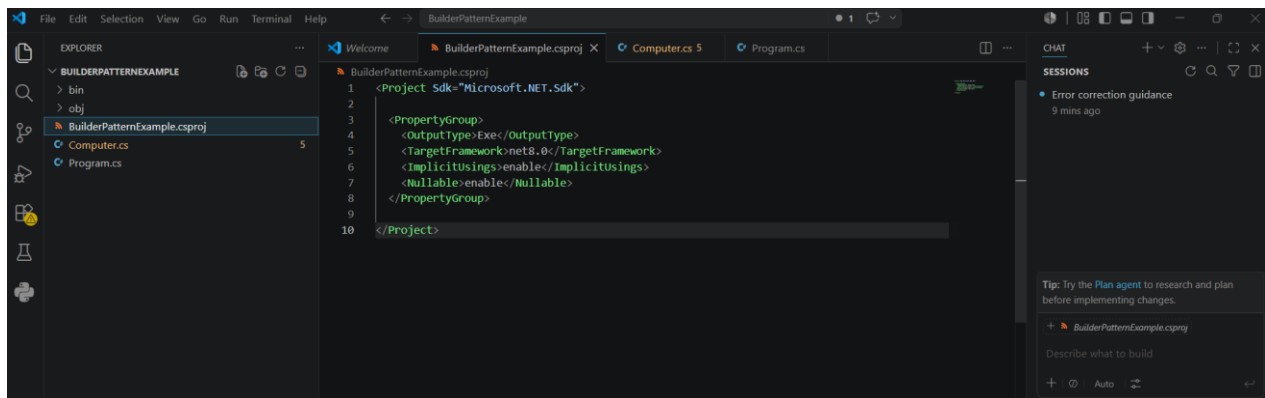
Exercise 3: Implementing the Builder Pattern

In this module, I was able to create, execute, and test the Builder Pattern in .NET using a project called BuilderPatternExample. I designed a Computer class with attributes like CPU, RAM, and Storage, and created a static nested ComputerBuilder class with methods to set each of these attributes, along with a Build() method to construct Computer objects. I also implemented a private constructor in the Computer class that accepts a Builder object to ensure that objects are created in a controlled and structured manner. After implementing the program, I executed it with different combinations of Computer objects to verify that the Builder Pattern worked correctly. This practical activity helped me understand how the Builder Pattern simplifies the creation of complex objects with optional parameters and improves code readability, flexibility, and maintainability in .NET applications.

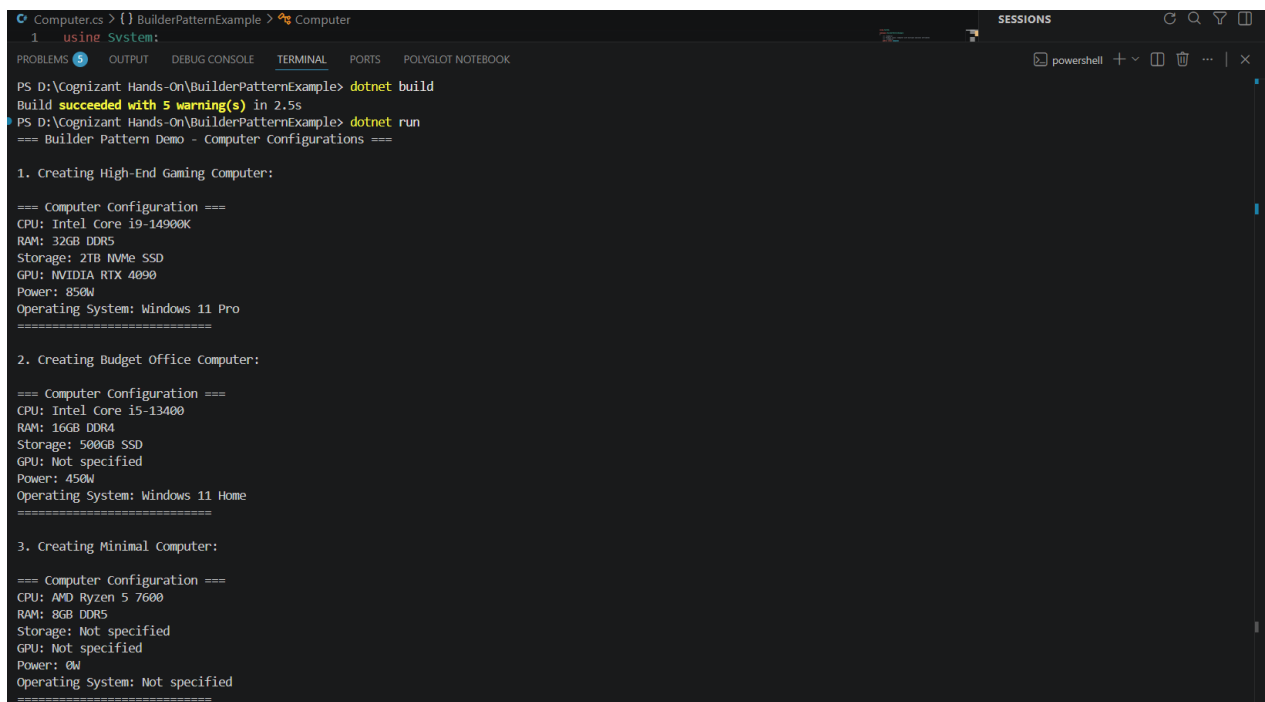
Coding:



```
1 namespace BuilderPatternExample
2 {
3     class Program
4     {
5         static void Main(string[] args)
6         {
7             Console.WriteLine("=== Builder Pattern Demo - Computer Configurations ===\n");
8
9             // Configuration 1: High-end Gaming Computer
10            Console.WriteLine("1. Creating High-End Gaming Computer:");
11            var gamingComputer = new Computer.Builder()
12                .WithCPU("Intel Core i9-14900K")
13                .WithRAM("32GB DDR5")
14                .WithStorage("2TB NVMe SSD")
15                .WithGPU("NVIDIA RTX 4090")
16                .WithPower(850)
17                .WithOperatingSystem("Windows 11 Pro")
18                .Build();
19            gamingComputer.DisplayDetails();
20
21            // Configuration 2: Budget Office Computer
22            Console.WriteLine("2. Creating Budget Office Computer:");
23            var officeComputer = new Computer.Builder()
24                .WithCPU("Intel Core i5-13400")
25                .WithRAM("16GB DDR4")
26                .WithStorage("500GB SSD")
27                .WithPower(450)
28                .WithOperatingSystem("Windows 11 Home")
29                .Build();
30            officeComputer.DisplayDetails();
31
32            // Configuration 3: Minimal Computer (only essential parts)
33            Console.WriteLine("3. Creating Minimal Computer:");
34            var minimalComputer = new Computer.Builder()
35                .WithCPU("AMD Ryzen 5 7600")
36                .WithRAM("8GB DDR5")
37                .Build();
38            minimalComputer.DisplayDetails();
39        }
40    }
41 }
```



Output:



```
Computer.cs > {} BuilderPatternExample > Computer
1 using System;

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POLYGLOT NOTEBOOK
PS D:\Cognizant Hands-On\BuilderPatternExample> dotnet run
Power: 450W
Operating System: Windows 11 Home
=====

3. Creating Minimal Computer:

=== Computer Configuration ===
CPU: AMD Ryzen 5 7600
RAM: 8GB DDR5
Storage: Not specified
GPU: Not specified
Power: 0W
Operating System: Not specified
=====

4. Creating Professional Workstation:

=== Computer Configuration ===
CPU: AMD Ryzen 9 7950X
RAM: 64GB DDR5
Storage: 4TB NVMe SSD + 8TB HDD
GPU: NVIDIA RTX 4080
Power: 1000W
Operating System: Ubuntu 24.04
=====

=== Builder Pattern Demo Complete ===
PS D:\Cognizant Hands-On\BuilderPatternExample>
```

Ln 8, Col 26 Spaces: 4 UTF-8 CRLF {} C#